

Inspecting, manipulating and visualising

a4a Training Course — Module 1.6

Danai Mantopoulou Palouka & Alessandro Ligas

2026-09-13

Three ways to look at an object

Way	Use it for
<code>summary()</code> , <code>print()</code>	dimensions, ranges, units — the quick check
<code>plot()</code>	the standard overview, one line
<code>as.data.frame()</code> → <code>ggplot()</code>	anything you want, exactly how you want it

The first two are free. The third is where you spend your time when building a report.

metrics() — the numbers people ask for

```
names(metrics(stk.fit))
```

```
[1] "Rec"      "SSB"      "Catch"    "Landings" "Discards" "F"        "HR"
```

Each is a method you can also call directly:

```
ssb(stk)    rec(stk)    fbar(stk)    catch(stk)
```

```
tsb(stk)    vb(stk)      fapex(stk)   sp(stk)
```

plot() is a method, not a function

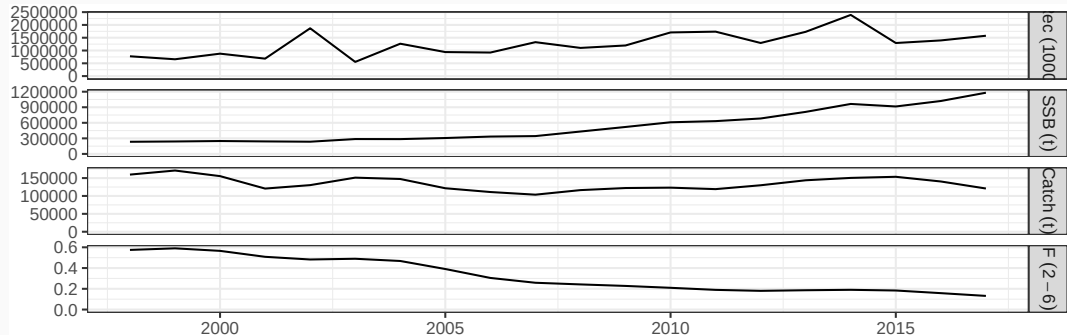
One name, different behaviour per class:

```
plot(flq)           # an FLQuant    - a single time series
plot(stk)           # an FLStock    - the four-panel overview
plot(idx)           # an FLIndex    - index by age
plot(FLStocks(...)) # several stocks - overlaid for comparison
```

Remember Module 1.1: `getMethod("plot", list("FLStock", "missing"))`
shows you exactly what it does.

The four-panel default

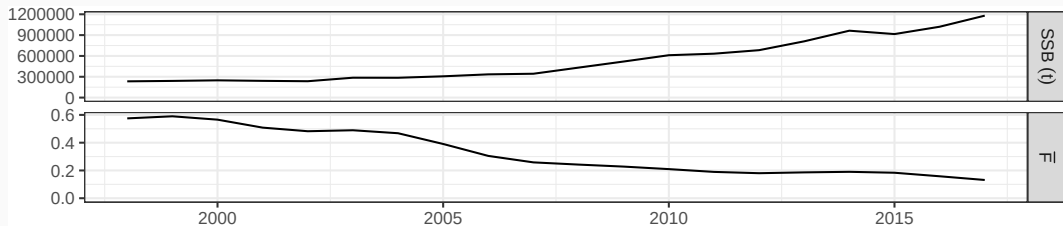
```
plot(stk.fit)
```



Change what it shows

The default came from that method's `metrics` argument — so override it:

```
plot(stk.fit, metrics = list(SSB = ssb, `bar(F)` = fbar))
```



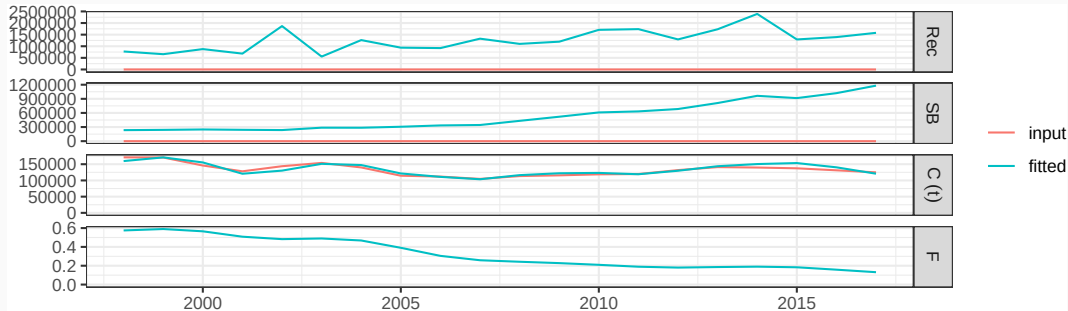
Panel names are plotmath — no spaces

Mean F errors. `Fbar` works. ``bar(F)`` renders as $\bar{F}$.

Comparing runs

FLStocks is how you put two assessments side by side:

```
plot(FLStocks(input = stk, fitted = stk.fit))
```



The escape hatch: `as.data.frame()`

Every FLR object converts to a long data.frame:

```
head(as.data.frame(catch.n(stk)), 3)
```

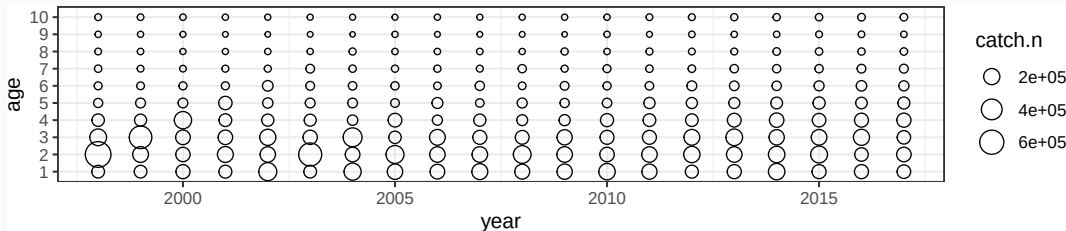
	age	year	unit	season	area	iter	data
1	1	1998	unique	all	unique	1	91657.58
2	2	1998	unique	all	unique	1	701954.20
3	3	1998	unique	all	unique	1	221681.40

From here it is ordinary `ggplot2` — data, year, age are just columns.

Bubble plots

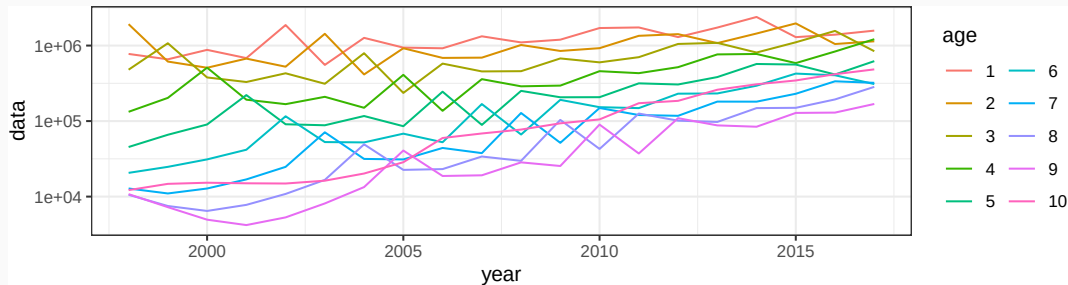
Size by value, split by age and year:

```
ggplot(catch.n(stk), aes(year, factor(age), size = data)) +  
  geom_point(shape = 21) + labs(y = "age", size = "catch.n")
```



Cohorts

```
ggplot(stock.n(stk.fit), aes(year, data, colour = factor(age))) +  
  geom_line() + scale_y_log10() + labs(colour = "age") +  
  guides(colour = guide_legend(ncol = 2))
```



Following a strong year class is a quick sanity check on the assessment.

Assembling figures

patchwork combines any ggplots with +, / and |:

```
p1 <- plot(ssb(stk.fit)) + ggtitle("SSB")
```

```
p2 <- plot(fbar(stk.fit)) + ggtitle("Mean F")
```

```
(p1 | p2) / plot(rec(stk.fit))
```

Useful on Day 5, when everything has to fit on one page.

Summary

- `summary()` for the check, `plot()` for the overview
- `metrics()` gives the standard quantities; each is also a method
- `plot()` takes a **metrics argument** — you choose the panels
- `FLStocks()` compares runs; you will use it constantly
- `as.data.frame()` then `ggplot2` for everything else

That is Day 1

Your data is in FLR, checked, and fitted. Tomorrow we start asking whether the fit is any good.

Exercise

Open `handson_M1.6_visualisation.R`.

1. `summary`, `units`, `metrics` and the individual methods
2. Subset and trim: `window()`, `trim()`, `[`
3. `plot()` per class, change its metrics, then **write your own metric**
4. `FLQuants`, faceting, log scales, and a 3D wireframe
5. `as.data.frame()` → bubbles, boxplots, cohorts
6. `geom_flpar` for reference points, and `cohcorrplot`
7. Assemble with `patchwork` and save to file